OpenMP — Practical Test Study Guide

Oct 14, 2026 · @Eduardo Kaluf

1. Summary (aulas 01–05)

Compile with `gcc -fopenmp`, set threads with `export OMP_NUM_THREADS=4`.

Core constructs

Construct	What it does	Implicit barrier at end?
`parallel`	Creates team; every thread runs the block (SPMD)	Yes (cannot remove)
`for`	Splits loop iterations among the team	Yes (`nowait` removes)
`parallel for`	Shortcut: both in one line	Yes
`sections` / `section`	Different block per thread (functional parallelism)	Yes (`nowait` removes)
`single`	One thread (any) runs the block	Yes (`nowait` removes)
`master` (now `masked`)	Only thread 0 runs the block	**No**
`critical [(name)]`	Mutual exclusion on a block; one thread at a time	—
`atomic`	Mutual exclusion on one scalar update (`x op= expr`, `x++`) — cheaper than critical	—
`barrier`	Every thread waits for all	—
`task` / `taskwait` / `taskgroup`	Deferred unit of work; wait for child tasks / whole group	—
`taskloop`	Loop chopped into tasks (`grainsize`, `num_tasks`)	Implicit taskgroup

Construct	What it does	Implicit barrier at end?
`simd`	Vectorize this loop (one thread, SIMD lanes)	—

Runtime library

- `omp_get_thread_num()` (ID),
- `omp_get_num_threads()` (team size — returns **1 outside** a parallel region),
- `omp_get_max_threads()`,
- `omp_set_num_threads(n)`,
- `omp_get_wtime()` (wall clock in seconds).
- Locks: `omp_init_lock`,
- `omp_set_lock` (blocks),
- `omp_unset_lock`,
- `omp_destroy_lock`;

Data scoping (the #1 source of bugs)

Defaults: variables declared **outside** the region, globals, `static` and heap data are **shared**. Variables declared **inside** the region, and locals of functions called from it, are **private**. Only the loop index of the `omp for` loop is made private automatically — **inner loop indices are NOT** in C.

Clause	Copy per thread?	Initial value inside	Value after region
`shared(x)`	No	original	whatever was last written
`private(x)`	Yes	**undefined**	original unchanged (unspecified if referenced)
`firstprivate(x)`	Yes	copied from original	original unchanged
`lastprivate(x)`	Yes	undefined	value from the **sequentially last** iteration

Clause	Copy per thread?	Initial value inside	Value after region
`reduction(op:x)`	Yes	identity of op (0 for +, 1 for *, ±max for min/max)	all copies combined into original

Loop scheduling

Schedule	Behavior	Use when
`static`	Contiguous blocks of ~N/T iterations, decided at compile/start time	Uniform cost per iteration; lowest overhead
`static,c`	Chunks of c handed out round-robin (c=1 → cyclic)	Cost grows/shrinks with i (triangular loops)
`dynamic,c`	Threads grab chunks of c from a queue	Unpredictable cost per iteration (Mandelbrot)
`guided,c`	Dynamic with shrinking chunks, down to c	Like dynamic, less queue overhead
`runtime`	Read from `OMP_SCHEDULE`	Testing schedules without recompiling
`auto`	Runtime decides	—

Manual work division

- Cyclic: `for (i = id; i < n; i += nthrds)` — fine for pure compute, bad for arrays (cache lines shared between threads).
- Block: `chunk = (n + nthrds - 1) / nthrds; start = id*chunk; end = min(start+chunk, n);` — best for arrays.

Synchronization cost, from cheapest to most expensive

No sharing (private + reduction) < `atomic` < `critical` < locks per element < `critical` inside a hot loop.

Put critical **after** the loop on a private partial sum, never around the `+=` inside it.

False sharing

Independent per-thread data on the **same 64-byte cache line** (e.g. `sum[id]` in a `double sum[NUM_THREADS]`) makes the line ping-pong between cores. Fixes: pad (`double sum[T][8]`), or better, accumulate in a private local variable and combine once at the end.

Tasks

Pattern: `parallel` → `single` (one producer) → `task` per unit of work; the other threads pick tasks up at the barrier. Use `firstprivate(p)` for the loop pointer, `shared(x)` for results, `taskwait` before using results, and a **cutoff** (`if(n > CUTOFF)` clause or a sequential fallback) for recursion.

SIMD

`#pragma omp simd` promises the compiler there are no loop-carried dependencies. Clauses: `reduction`, `private`, `safelen(k)` (dependency distance ≥ k is safe), `aligned(p:32)`, `linear(i:1)`, `collapse`. `for simd` = threads + vectors. Compile with `-fopenmp-simd` (SIMD only) or `-fopenmp`.

NUMA / affinity

`proc_bind(close)` packs threads near the master (data locality);

`spread` scatters them across sockets (memory bandwidth);

`master` puts them on the master's place.

2. Can it be parallelized? How to justify

A loop can run in parallel when its iterations can execute in **any order** and give the same result. Justify by naming the dependency (or its absence) and the variable that carries it.

Method for any loop

1. List every variable written in the loop body.
2. For each, ask: is it written in iteration i and read/written in a **different** iteration j? That is a loop-carried dependency.
3. Classify it and pick the fix from the table below.

4. Check the loop shape: `omp for` needs a countable loop (`for (i = a; i < b; i += c)` , bounds fixed before the loop, no `break`). `while` loops and pointer chasing need tasks or conversion to an array.
5. Check that the work is big enough: overhead of creating/scheduling threads (~µs) must be small vs the loop's cost.

Pattern	Dependency	Parallel?	Fix
`a[i] = b[i] + c[i]`	None	Yes	`parallel for`
`sum += a[i]`	Accumulator (associative op)	Yes	`reduction(+:sum)`
`if (a[i] > m) m = a[i]`	Accumulator	Yes	`reduction(max:m)`
`x = f(i); a[i] = x*x`	`x` reused (false dep, only scoping)	Yes	`private(x)` or declare inside
`j += 2; a[i] = g(j)`	Induction variable	Yes, after rewrite	`j = 5 + 2*(i+1)` computed from i
`a[i] = a[i-1] + b[i]`	True (RAW), distance 1	**No** as written	Sequential, or a parallel scan algorithm
`a[i] = a[i+1] + b[i]`	Anti (WAR)	Yes, with a copy	Read from `a_old` copy
`a[i] = a[i-4] + b[i]`	RAW, distance 4	No for threads; yes for SIMD	`simd safelen(4)`
`hist[key[i]]++`	Possible conflict on same element	Yes, with sync	atomic / locks / private copies / array reduction
`if (a[i]==target) { pos=i; break; }`	Control (early exit)	Not with `omp for`	Flag + no break, or `cancel for`
`while (p) { work(p); p=p->next; }`	Traversal is sequential, work is not	Work only	Array of pointers + `for` , or `single` + `task firstprivate(p)`
Calls `rand()` / `printf` / shared buffer	Hidden shared state	Careful	`rand_r(&seed)` per thread, accept unordered output

3. Debugging checklist

1. **Inner loop indices shared.** `#pragma omp parallel for` over `i` with `int j` declared outside → `j` is shared. Fix: `private(j)` or `for (int j = …)`.
2. **Temporaries shared.** `x`, `tmp`, `sum`, a struct like `c` declared outside the region → private.
3. **Globals used by called functions.** `private(c)` makes a private copy in `main`, but a function that reads the **global** `c` still sees the shared one. Fix: pass it as a parameter.
4. **`private` used where `firstprivate` was needed.** Private copies start uninitialized (`eps`, `tmp = 0`).
5. **Value needed after the region.** Private copies are discarded → `reduction` or `lastprivate`.
6. **Unprotected updates.** `count++`, `sum +=`, `hist[k]++` on shared data → reduction, atomic or critical.
7. **Missing barrier.** `nowait` or `master` (no implicit barrier) followed by code that reads what other threads wrote.
8. **Called outside the region.** `omp_get_num_threads()` before `#pragma omp parallel` returns 1.
9. **Several threads doing work meant for one.** Task creation, `taskloop`, initialization or `printf` without `single`.
10. **Tasks:** results lost (no `shared`), used before ready (no `taskwait`), pointer race (no `firstprivate(p)`), no cutoff (millions of tiny tasks).
11. **Nested parallel by accident.** `parallel` inside `parallel` → T×T threads, or nested regions serialized.
12. **Correct but slow.** `critical` inside the hot loop, false sharing on `sum[id]`, wrong schedule for unbalanced work, region too small for the overhead.

Tools and tricks

- `gcc -fopenmp -g -fsanitize=thread` reports data races (may give false positives inside the OpenMP runtime).
- `-fopt-info-vec-missed` explains why a loop was not vectorized.

4. Timing and speedup

Measure only the region of interest with `omp_get_wtime()` (wall clock), repeat runs, and report speedup against the **sequential** time, not the 1-thread parallel version.

```
double t0 = omp_get_wtime();
/* region of interest only: no malloc, no init, no printf */
double t1 = omp_get_wtime();
printf("%d threads: %.4f s\n", omp_get_max_threads(), t1 - t0);
```

```
gcc -O3 -fopenmp prog.c -o prog
for t in 1 2 4 8; do OMP_NUM_THREADS=$t ./prog; done
```

5. Class exercises, solved

5a. Mandelbrot (exercise 6) — four bugs

#	Bug	Why it breaks	Fix
1	`testpoint()` reads the **global** `c`	`private(c)` only privatizes `c` inside `main`'s region; the function still reads the shared global, which other threads overwrite	Pass `c` as a parameter: `testpoint(struct d_complex c)`
2	`numoutside++` on a global	Unprotected read-modify-write → lost updates	Return 0/1 and use `reduction(+:numoutside)` (or `#pragma omp atomic`)
3	`j` is shared	Only `i` (the `omp for` index) is private by default	`private(j)` or `for (int j = …)`
4	`private(eps)`	Private copy is uninitialized → garbage added to `c.r`, `c.i`	Leave `eps` shared (read-only) or `firstprivate(eps)`

Optimization: points inside the set run all 1000 iterations, points outside exit early, so cost per `i` is very uneven → `schedule(dynamic)` or `schedule(guided)` beats the default `static`.

```
static int testpoint(struct d_complex c) {
    struct d_complex z = c; double temp;
    for (int iter = 0; iter < MAXITER; iter++) {
        temp = z.r*z.r - z.i*z.i + c.r;
        z.i  = z.r*z.i*2 + c.i;
        z.r  = temp;
        if (z.r*z.r + z.i*z.i > 4.0) return 1;   /* outside */
    }
    return 0;
}

int main(void) {
    int numoutside = 0; double eps = 1.0e-5;
    #pragma omp parallel for default(none) shared(eps) \
                             reduction(+:numoutside) schedule(dynamic)
    for (int i = 0; i < NPOINTS; i++)
        for (int j = 0; j < NPOINTS; j++) {
            struct d_complex c;
            c.r = -2.0 + 2.5*(double)i/NPOINTS + eps;
            c.i = 1.125*(double)j/NPOINTS + eps;
            numoutside += testpoint(c);
        }
    /* area computation unchanged */
}
```

5b. Linked list

Version 1, no tasks — copy pointers to an array:

```
int count = 0;
for (p = head; p; p = p->next) count++;
struct node **parr = malloc(count * sizeof *parr);
p = head;
for (int i = 0; i < count; i++) { parr[i] = p; p = p->next; }

#pragma omp parallel for schedule(dynamic,1)
for (int i = 0; i < count; i++)
    processwork(parr[i]);
free(parr);
```

Version 2, tasks:

```
#pragma omp parallel
#pragma omp single
{
    for (struct node *q = head; q; q = q->next) {
        #pragma omp task firstprivate(q)
        processwork(q);
    }
}   /* implicit barrier: all tasks done here */
```

5c. SIMD (`process_simd`)

```
void process_simd(const float *restrict x, const float *restrict y,
                  float *restrict z, int n, float alpha, float beta) {
    #pragma omp simd
    for (int i = 0; i < n; i++) {
        float v = alpha * x[i] + beta * y[i];
        z[i] = (v > 0.0f) ? v * v : 0.0f;   /* branch → masked select / blend
*/
    }
}
```

Why it is vectorizable: each `z[i]` depends only on `x[i]` and `y[i]`, no loop-carried dependency; the `if` becomes a compare + blend, so all lanes run the same instruction. `restrict` tells the compiler the arrays don't overlap.